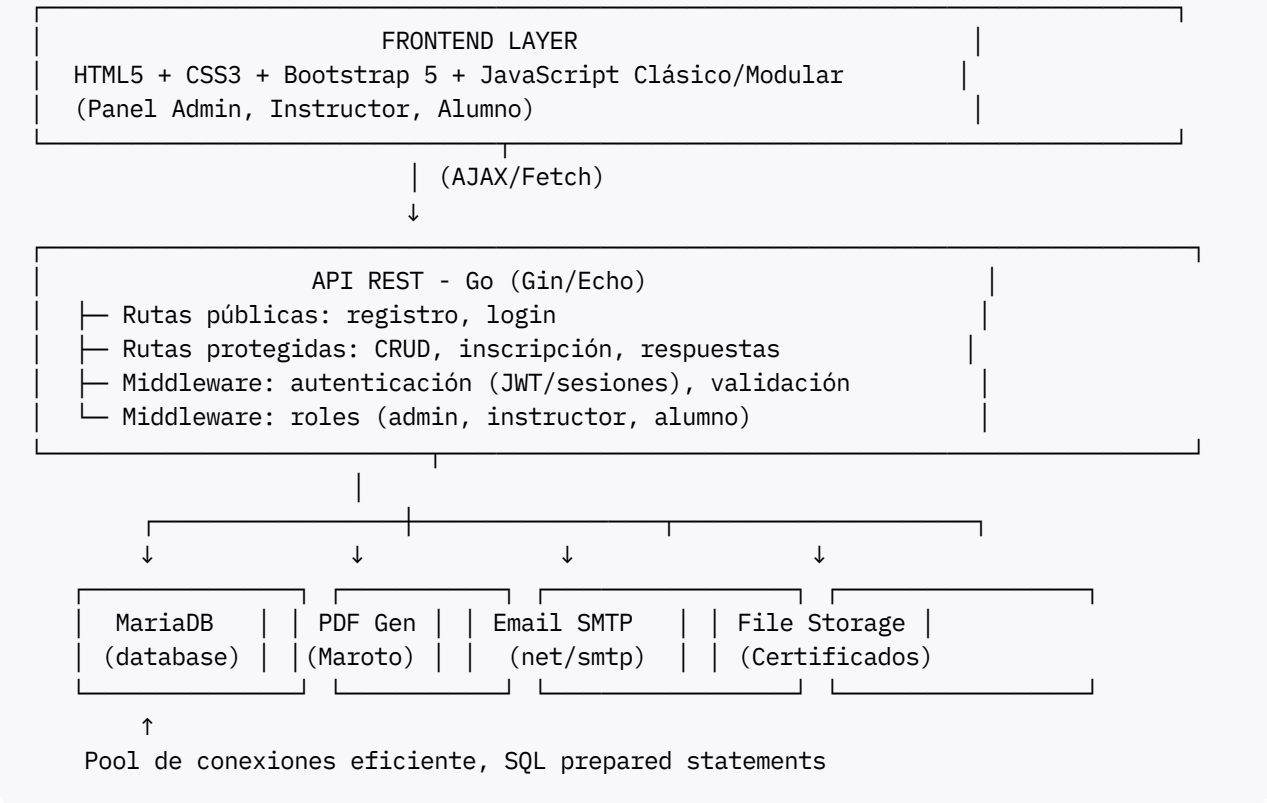
Aplicación Web de Gestión de Cursos Online con Go, MariaDB y Bootstrap

Te presento un diseño completo y detallado de tu plataforma educativa. Cubriré arquitectura, modelo de datos, implementación backend, frontend y flujos de negocio críticos.

1. ARQUITECTURA GENERAL



Flujo de datos:

1. Frontend envía solicitudes AJAX a la API REST
2. Go valida, consulta MariaDB, aplica lógica de negocio
3. Generación de PDFs en memoria o disco
4. Envío de emails vía SMTP con attachments
5. Logs centralizados para auditoría

2. MODELO DE DATOS EN MARIADB

Esquema completo en SQL:

```
-- ============================================================================
-- TABLA: usuario (autenticación y roles)
-- ============================================================================
CREATE TABLE usuario (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    rol ENUM('administrador', 'instructor', 'alumno') NOT NULL DEFAULT 'alumno',
    hash_password VARCHAR(255) NOT NULL,
    activo BOOLEAN DEFAULT TRUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_email (email),
    INDEX idx_rol (rol)
);

-- ============================================================================
-- TABLA: curso (cursos disponibles)
-- ============================================================================
CREATE TABLE curso (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(255) NOT NULL,
    descripcion TEXT,
    instructor_id INT NOT NULL,
    tipo ENUM('toma_conocimientos', 'test', 'encuesta', 'mixto') NOT NULL DEFAULT 'mixto'
    fecha_inicio DATETIME,
    fecha_fin DATETIME,
    dias_para_completar INT DEFAULT NULL,
    nota_minima DECIMAL(5, 2) DEFAULT 70.00,
    activo BOOLEAN DEFAULT TRUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (instructor_id) REFERENCES usuario(id) ON DELETE CASCADE,
    INDEX idx_instructor (instructor_id),
    INDEX idx_tipo (tipo)
);

-- ============================================================================
-- TABLA: inscripcion_curso (relación alumno-curso)
-- ============================================================================
CREATE TABLE inscripcion_curso (
    id INT AUTO_INCREMENT PRIMARY KEY,
    alumno_id INT NOT NULL,
    curso_id INT NOT NULL,
    fecha_inscripcion DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_limite DATETIME,
    estado ENUM('en_curso', 'aprobado', 'reprobado', 'plazo_vencido') DEFAULT 'en_curso',
    porcentaje_avance DECIMAL(5, 2) DEFAULT 0.00,
    fecha_finalizacion DATETIME DEFAULT NULL,
    FOREIGN KEY (alumno_id) REFERENCES usuario(id) ON DELETE CASCADE,
    FOREIGN KEY (curso_id) REFERENCES curso(id) ON DELETE CASCADE,
    UNIQUE KEY unique_alumno_curso (alumno_id, curso_id),
```

```
    INDEX idx_estado (estado),
    INDEX idx_alumno (alumno_id)
);

-- ============================================================================
-- TABLA: modulo (módulos dentro de un curso)
-- ============================================================================
CREATE TABLE modulo (
    id INT AUTO_INCREMENT PRIMARY KEY,
    curso_id INT NOT NULL,
    titulo VARCHAR(255) NOT NULL,
    descripcion TEXT,
    orden INT NOT NULL,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (curso_id) REFERENCES curso(id) ON DELETE CASCADE,
    INDEX idx_curso (curso_id),
    UNIQUE KEY unique_curso_orden (curso_id, orden)
);

-- ============================================================================
-- TABLA: actividad (contenidos, tests, encuestas)
-- ============================================================================
CREATE TABLE actividad (
    id INT AUTO_INCREMENT PRIMARY KEY,
    modulo_id INT NOT NULL,
    tipo_actividad ENUM('contenido', 'test', 'encuesta', 'toma_conocimiento') NOT NULL,
    titulo VARCHAR(255) NOT NULL,
    descripcion TEXT,
    contenido_html TEXT,
    orden INT NOT NULL,
    obligatoria BOOLEAN DEFAULT TRUE,
    es_evaluable BOOLEAN DEFAULT FALSE,
    tiempo_limite_minutos INT DEFAULT NULL,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (modulo_id) REFERENCES modulo(id) ON DELETE CASCADE,
    INDEX idx_modulo (modulo_id),
    UNIQUE KEY unique_modulo_orden (modulo_id, orden)
);

-- ============================================================================
-- TABLA: pregunta (preguntas para tests y encuestas)
-- ============================================================================
CREATE TABLE pregunta (
    id INT AUTO_INCREMENT PRIMARY KEY,
    actividad_id INT NOT NULL,
    texto_pregunta TEXT NOT NULL,
    tipo ENUM('opcion_multiple', 'verdadero_falso', 'respuesta_abierta', 'escala_likert')
    orden INT NOT NULL,
    es_obligatoria BOOLEAN DEFAULT TRUE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (actividad_id) REFERENCES actividad(id) ON DELETE CASCADE,
    INDEX idx_actividad (actividad_id),
    UNIQUE KEY unique_actividad_orden (actividad_id, orden)
);

-- ============================================================================
```

```
-- TABLA: opcion_respuesta (opciones para preguntas múltiples)
-- ============================================================================
CREATE TABLE opcion_respuesta (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pregunta_id INT NOT NULL,
    texto_opcion VARCHAR(255) NOT NULL,
    es_correcta BOOLEAN DEFAULT FALSE,
    orden INT NOT NULL,
    puntos DECIMAL(5, 2) DEFAULT 1.00,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (pregunta_id) REFERENCES pregunta(id) ON DELETE CASCADE,
    INDEX idx_pregunta (pregunta_id),
    UNIQUE KEY unique_pregunta_orden (pregunta_id, orden)
);

-- ============================================================================
-- TABLA: intento_actividad (registro de intentos del alumno)
-- ============================================================================
CREATE TABLE intento_actividad (
    id INT AUTO_INCREMENT PRIMARY KEY,
    inscripcion_curso_id INT NOT NULL,
    actividad_id INT NOT NULL,
    fecha_inicio DATETIME DEFAULT CURRENT_TIMESTAMP,
    fecha_finalizacion DATETIME DEFAULT NULL,
    calificacion DECIMAL(5, 2) DEFAULT NULL,
    completada BOOLEAN DEFAULT FALSE,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (inscripcion_curso_id) REFERENCES inscripcion_curso(id) ON DELETE CASCADE
    FOREIGN KEY (actividad_id) REFERENCES actividad(id) ON DELETE CASCADE,
    INDEX idx_inscripcion (inscripcion_curso_id),
    INDEX idx_actividad (actividad_id)
);

-- ============================================================================
-- TABLA: respuesta_pregunta (respuestas individuales)
-- ============================================================================
CREATE TABLE respuesta_pregunta (
    id INT AUTO_INCREMENT PRIMARY KEY,
    intento_actividad_id INT NOT NULL,
    pregunta_id INT NOT NULL,
    opcion_respuesta_id INT DEFAULT NULL,
    texto_respuesta TEXT DEFAULT NULL,
    puntos_obtenidos DECIMAL(5, 2) DEFAULT 0.00,
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (intento_actividad_id) REFERENCES intento_actividad(id) ON DELETE CASCADE
    FOREIGN KEY (pregunta_id) REFERENCES pregunta(id) ON DELETE CASCADE,
    FOREIGN KEY (opcion_respuesta_id) REFERENCES opcion_respuesta(id) ON DELETE SET NULL,
    INDEX idx_intento (intento_actividad_id)
);

-- ============================================================================
-- TABLA: certificado (PDF de certificación)
-- ============================================================================
CREATE TABLE certificado (
    id INT AUTO_INCREMENT PRIMARY KEY,
    alumno_id INT NOT NULL,
```

```
    curso_id INT NOT NULL,
    inscripcion_curso_id INT NOT NULL,
    fecha_emision DATETIME DEFAULT CURRENT_TIMESTAMP,
    codigo_unico VARCHAR(64) UNIQUE NOT NULL,
    ruta_pdf VARCHAR(500) NOT NULL,
    calificacion_final DECIMAL(5, 2),
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (alumno_id) REFERENCES usuario(id) ON DELETE CASCADE,
    FOREIGN KEY (curso_id) REFERENCES curso(id) ON DELETE CASCADE,
    FOREIGN KEY (inscripcion_curso_id) REFERENCES inscripcion_curso(id) ON DELETE CASCADE
    INDEX idx_alumno (alumno_id),
    INDEX idx_codigo (codigo_unico),
    INDEX idx_curso (curso_id)
);

-- ============================================================================
-- TABLA: log_envio_email (auditoría de emails)
-- ============================================================================
CREATE TABLE log_envio_email (
    id INT AUTO_INCREMENT PRIMARY KEY,
    destinatario VARCHAR(255) NOT NULL,
    asunto VARCHAR(255) NOT NULL,
    tipo_evento ENUM('aprobacion', 'plazo_vencido', 'cambio_estado', 'otro') NOT NULL,
    certificado_id INT DEFAULT NULL,
    fecha_envio DATETIME DEFAULT CURRENT_TIMESTAMP,
    estado ENUM('enviado', 'fallido', 'pendiente') DEFAULT 'pendiente',
    mensaje_error TEXT DEFAULT NULL,
    INDEX idx_destinatario (destinatario),
    INDEX idx_tipo_evento (tipo_evento),
    FOREIGN KEY (certificado_id) REFERENCES certificado(id) ON DELETE SET NULL
);

-- ============================================================================
-- Índices adicionales para optimización
-- ============================================================================
CREATE INDEX idx_log_fecha ON log_envio_email(fecha_envio);
CREATE INDEX idx_inscripcion_estado ON inscripcion_curso(estado, fecha_limite);
CREATE INDEX idx_intento_completa ON intento_actividad(completada, fecha_finalizacion);
```

Notas sobre el modelo:

- **Flexibilidad de cursos**: Cada curso puede ser de tipo `toma_conocimientos`, `test`, `encuesta` o `mixto`
- **Módulos y actividades**: Cursos contienen módulos; módulos contienen actividades
- **Actividades heterogéneas**: Una misma actividad puede ser contenido, test, encuesta, etc.
- **Intentos y respuestas**: Registro completo de cada intento del alumno para auditoría
- **Certificados**: Vinculados a inscripciones, con código único y ruta de archivo
- **Logs de email**: Para rastrear fallos y reintentaos

3. BACKEND EN GO

3.1. Estructura de carpetas

```
course-platform/
├── main.go
├── go.mod
├── go.sum
├── config/
│   └── config.go
├── models/
│   ├── usuario.go
│   ├── curso.go
│   ├── modulo.go
│   ├── actividad.go
│   ├── inscripcion.go
│   ├── certificado.go
│   └── email.go
├── handlers/
│   ├── auth.go
│   ├── curso.go
│   ├── modulo.go
│   ├── actividad.go
│   ├── inscripcion.go
│   ├── progreso.go
│   └── certificado.go
├── services/
│   ├── usuario_service.go
│   ├── curso_service.go
│   ├── progreso_service.go
│   ├── certificado_service.go
│   └── email_service.go
├── middleware/
│   ├── auth.go
│   └── role.go
├── db/
│   ├── db.go
│   └── migrations.sql
├── utils/
│   ├── jwt.go
│   ├── password.go
│   ├── validator.go
│   └── logger.go
├── templates/
│   ├── email_aprobacion.html
│   ├── email_plazo_vencido.html
│   └── certificado_template.html
└── cron/
    └── scheduler.go
```

3.2. Archivo `go.mod` y dependencias

```
module course-platform

go 1.21

require (
    github.com/gin-gonic/gin v1.9.1
    github.com/go-sql-driver/mysql v1.7.1
    github.com/golang-jwt/jwt/v5 v5.0.0
    github.com/joho/godotenv v1.5.1
    github.com/jung-kurt/gofpdf/v2 v2.16.2
    golang.org/x/crypto v0.14.0
    github.com/robfig/cron/v3 v3.0.1
)
```

3.3. Configuración (`config/config.go`)

```
package config

import (
    "log"
    "os"

    "github.com/joho/godotenv"
)

type AppConfig struct {
    DBHost         string
    DBPort         string
    DBUser         string
    DBPassword     string
    DBName         string
    JWTSecret      string
    SMTPHost       string
    SMTPPort       string
    SMTPUser       string
    SMTPPassword   string
    EmailFrom      string
    CertDir        string
    ServerPort     string
    Environment    string
}

func LoadConfig() *AppConfig {
    godotenv.Load()

    return &AppConfig{
        DBHost:        getEnv("DB_HOST", "localhost"),
        DBPort:        getEnv("DB_PORT", "3306"),
        DBUser:        getEnv("DB_USER", "root"),
        DBPassword:    getEnv("DB_PASSWORD", ""),
        DBName:        getEnv("DB_NAME", "course_db"),
        JWTSecret:     getEnv("JWT_SECRET", "your-secret-key"),
```

```
        SMTPHost:      getEnv("SMTP_HOST", "smtp.gmail.com"),
        SMTPPort:      getEnv("SMTP_PORT", "587"),
        SMTPUser:      getEnv("SMTP_USER", ""),
        SMTPPassword:  getEnv("SMTP_PASSWORD", ""),
        EmailFrom:     getEnv("EMAIL_FROM", "noreply@courseplatform.com"),
        CertDir:       getEnv("CERT_DIR", "./certs"),
        ServerPort:    getEnv("SERVER_PORT", "8080"),
        Environment:   getEnv("ENVIRONMENT", "development"),
    }
}

func getEnv(key, defaultValue string) string {
    if value := os.Getenv(key); value != "" {
        return value
    }
    return defaultValue
}
```

3.4. Conexión a MariaDB (`db/db.go`)

```
package db

import (
    "database/sql"
    "fmt"
    "log"
    "time"

    _ "github.com/go-sql-driver/mysql"
    "course-platform/config"
)

var DB *sql.DB

func InitDB(cfg *config.AppConfig) error {
    dsn := fmt.Sprintf(
        "%s:%s@tcp(%s:%s)/%s?parseTime=true&loc=UTC",
        cfg.DBUser,
        cfg.DBPassword,
        cfg.DBHost,
        cfg.DBPort,
        cfg.DBName,
    )

    var err error
    DB, err = sql.Open("mysql", dsn)
    if err != nil {
        return fmt.Errorf("error al abrir BD: %w", err)
    }

    // Pool de conexiones
    DB.SetMaxOpenConns(25)
    DB.SetMaxIdleConns(5)
    DB.SetConnMaxLifetime(5 * time.Minute)
```

```
    // Ping para validar conexión
    if err := DB.Ping(); err != nil {
        return fmt.Errorf("error al conectar a BD: %w", err)
    }

    log.Println("✓ Conexión a MariaDB exitosa")
    return nil
}

func CloseDB() error {
    if DB != nil {
        return DB.Close()
    }
    return nil
}
```

3.5. Modelos (`models/usuario.go`, `models/curso.go`, etc.)

models/usuario.go:

```
package models

import "time"

type Usuario struct {
    ID           int       `json:"id"`
    Nombre       string    `json:"nombre"`
    Email        string    `json:"email"`
    Rol          string    `json:"rol"` // administrador, instructor, alumno
    HashPassword string    `json:"-"`
    Activo       bool      `json:"activo"`
    FechaCreacion time.Time `json:"fecha_creacion"`
}

type RegistroRequest struct {
    Nombre   string `json:"nombre" binding:"required,min=3"`
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required,min=8"`
    Rol      string `json:"rol" binding:"required,oneof=alumno instructor"`
}

type LoginRequest struct {
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required"`
}

type LoginResponse struct {
    Token   string  `json:"token"`
    Usuario Usuario `json:"usuario"`
}
```

models/curso.go:

```
package models

import "time"

type Curso struct {
    ID               int       `json:"id"`
    Titulo           string    `json:"titulo"`
    Descripcion      string    `json:"descripcion"`
    InstructorID     int       `json:"instructor_id"`
    Tipo             string    `json:"tipo"` // toma_conocimientos, test, encuesta, mixto
    FechaInicio      time.Time `json:"fecha_inicio"`
    FechaFin         *time.Time `json:"fecha_fin"`
    DiasParaCompletar *int      `json:"dias_para_completar"`
    NotaMinima       float64   `json:"nota_minima"`
    Activo           bool      `json:"activo"`
    FechaCreacion    time.Time `json:"fecha_creacion"`
}

type CreateCursoRequest struct {
    Titulo            string    `json:"titulo" binding:"required,min=5"`
    Descripcion       string    `json:"descripcion"`
    Tipo              string    `json:"tipo" binding:"required"`
    FechaInicio       time.Time `json:"fecha_inicio"`
    FechaFin          *time.Time `json:"fecha_fin"`
    DiasParaCompletar *int      `json:"dias_para_completar"`
    NotaMinima        float64   `json:"nota_minima"`
}
```

models/inscripcion.go:

```
package models

import "time"

type InscripcionCurso struct {
    ID                  int       `json:"id"`
    AlumnoID            int       `json:"alumno_id"`
    CursoID             int       `json:"curso_id"`
    FechaInscripcion    time.Time `json:"fecha_inscripcion"`
    FechaLimite         *time.Time `json:"fecha_limite"`
    Estado              string    `json:"estado"` // en_curso, aprobado, reprobado, plazo
    PorcentajeAvance    float64   `json:"porcentaje_avance"`
    FechaFinalizacion   *time.Time `json:"fecha_finalizacion"`
}
```

models/certificado.go:

```
package models

import "time"

type Certificado struct {
    ID                 int       `json:"id"`
```

```
    AlumnoID           int       `json:"alumno_id"`
    CursoID            int       `json:"curso_id"`
    InscripcionCursoID int       `json:"inscripcion_curso_id"`
    FechaEmision       time.Time `json:"fecha_emision"`
    CodigoUnico        string    `json:"codigo_unico"`
    RutaPDF            string    `json:"ruta_pdf"`
    CalificacionFinal  float64   `json:"calificacion_final"`
}
```

3.6. Servicios principales

services/progreso_service.go:

```
package services

import (
    "database/sql"
    "fmt"
    "time"

    "course-platform/db"
    "course-platform/models"
)

type ProgresoService struct {
    DB *sql.DB
}

func NewProgresoService() *ProgresoService {
    return &ProgresoService{DB: db.DB}
}

// CalcularPorcentajeAvance calcula el progreso de un alumno en un curso
func (s *ProgresoService) CalcularPorcentajeAvance(inscripcionID int) (float64, error) {
    query := `
        SELECT COUNT(DISTINCT a.id) as total_actividades,
               SUM(CASE WHEN ia.completada = TRUE THEN 1 ELSE 0 END) as completadas
        FROM actividad a
        JOIN modulo m ON a.modulo_id = m.id
        JOIN inscripcion_curso ic ON m.curso_id = ic.curso_id
        LEFT JOIN intento_actividad ia ON a.id = ia.actividad_id
                                       AND ia.inscripcion_curso_id = ic.id
        WHERE ic.id = ?
          AND a.obligatoria = TRUE
    `

    var totalActividades, completadas sql.NullInt64
    err := s.DB.QueryRow(query, inscripcionID).Scan(&totalActividades, &completadas)
    if err != nil {
        return 0, err
    }

    if !totalActividades.Valid || totalActividades.Int64 == 0 {
        return 0, nil
    }
```

```
    porcentaje := float64(completadas.Int64) / float64(totalActividades.Int64) * 100
    return porcentaje, nil
}

// EvaluarAprobacion determina si un alumno aprobó según el tipo de curso
func (s *ProgresoService) EvaluarAprobacion(inscripcionID int) (bool, float64, error) {
    // Obtener curso y nota mínima
    query := `
        SELECT c.tipo, c.nota_minima,
               COALESCE(AVG(ia.calificacion), 0) as promedio_calificacion
        FROM inscripcion_curso ic
        JOIN curso c ON ic.curso_id = c.id
        LEFT JOIN intento_actividad ia ON ic.id = ia.inscripcion_curso_id
                                       AND ia.es_evaluable = TRUE
        WHERE ic.id = ?
        GROUP BY c.id
    `

    var tiposCurso, notaMinima, promedioCalificacion sql.NullFloat64
    err := s.DB.QueryRow(query, inscripcionID).Scan(&tiposCurso, &notaMinima, &promedioCa
    if err != nil {
        return false, 0, err
    }

    // Si no hay calificación registrada, no aprobó
    if !promedioCalificacion.Valid {
        promedioCalificacion.Float64 = 0
    }

    // Determinar aprobación según tipo de curso
    aprobado := promedioCalificacion.Float64 >= notaMinima.Float64
    return aprobado, promedioCalificacion.Float64, nil
}

// VerificarPlazo verifica si el alumno está dentro del plazo
func (s *ProgresoService) VerificarPlazo(inscripcionID int) (bool, error) {
    query := `
        SELECT ic.fecha_limite, ic.fecha_inscripcion, c.dias_para_completar
        FROM inscripcion_curso ic
        JOIN curso c ON ic.curso_id = c.id
        WHERE ic.id = ?
    `

    var fechaLimite sql.NullTime
    var fechaInscripcion time.Time
    var diasParaCompletar sql.NullInt64

    err := s.DB.QueryRow(query, inscripcionID).Scan(&fechaLimite, &fechaInscripcion, &dia
    if err != nil {
        return false, err
    }

    now := time.Now()

    // Caso 1: Fecha límite explícita
```

```
    if fechaLimite.Valid {
        return now.Before(fechaLimite.Time), nil
    }

    // Caso 2: Días para completar desde inscripción
    if diasParaCompletar.Valid {
        fechaLimiteCalculada := fechaInscripcion.AddDate(0, 0, int(diasParaCompletar.Int6
        return now.Before(fechaLimiteCalculada), nil
    }

    // Caso 3: Sin límite de plazo
    return true, nil
}

// ActualizarEstadoInscripcion actualiza el estado según aprobación y plazo
func (s *ProgresoService) ActualizarEstadoInscripcion(inscripcionID int) error {
    aprobado, _, err := s.EvaluarAprobacion(inscripcionID)
    if err != nil {
        return err
    }

    dentroDelPlazo, err := s.VerificarPlazo(inscripcionID)
    if err != nil {
        return err
    }

    var nuevoEstado string
    if !dentroDelPlazo {
        nuevoEstado = "plazo_vencido"
    } else if aprobado {
        nuevoEstado = "aprobado"
    } else {
        nuevoEstado = "en_curso"
    }

    updateQuery := `
        UPDATE inscripcion_curso
        SET estado = ?, fecha_finalizacion = ?
        WHERE id = ?
    `

    var fechaFin *time.Time
    if nuevoEstado == "aprobado" || nuevoEstado == "plazo_vencido" {
        now := time.Now()
        fechaFin = &now
    }

    _, err = s.DB.Exec(updateQuery, nuevoEstado, fechaFin, inscripcionID)
    return err
}

// CalcularCalificacionFinal obtiene la nota final del curso
func (s *ProgresoService) CalcularCalificacionFinal(inscripcionID int) (float64, error) {
    query := `
        SELECT COALESCE(AVG(ia.calificacion), 0)
        FROM intento_actividad ia
```

```
        WHERE ia.inscripcion_curso_id = ?
          AND ia.es_evaluable = TRUE
    `

    var calificacion float64
    err := s.DB.QueryRow(query, inscripcionID).Scan(&calificacion)
    return calificacion, err
}
```

services/certificado_service.go:

```
package services

import (
    "crypto/rand"
    "encoding/hex"
    "fmt"
    "os"
    "path/filepath"
    "time"

    "github.com/jung-kurt/gofpdf/v2"
    "course-platform/db"
    "course-platform/models"
)

type CertificadoService struct {
    DB      *sql.DB
    CertDir string
}

func NewCertificadoService(certDir string) *CertificadoService {
    return &CertificadoService{DB: db.DB, CertDir: certDir}
}

// GenerarCodigoUnico genera un código único para el certificado
func (s *CertificadoService) GenerarCodigoUnico() (string, error) {
    bytes := make([]byte, 16)
    if _, err := rand.Read(bytes); err != nil {
        return "", err
    }
    return hex.EncodeToString(bytes), nil
}

// GenerarPDFCertificado crea un PDF personalizado del certificado
func (s *CertificadoService) GenerarPDFCertificado(
    nombreAlumno string,
    tituloCurso string,
    fechaFinalizacion time.Time,
    codigoUnico string,
    calificacion float64,
) (string, error) {
    // Crear directorio si no existe
    if err := os.MkdirAll(s.CertDir, 0755); err != nil {
        return "", err
```

```
    }

    // Crear PDF
    pdf := gofpdf.New("P", "mm", "A4", "")
    pdf.AddPage()
    pdf.SetFont("Arial", "", 12)

    // Fondo (simulado con rectángulo de borde)
    pdf.SetLineWidth(2)
    pdf.Rect(10, 10, 190, 277, "D")

    // Título
    pdf.SetFont("Arial", "B", 28)
    pdf.SetTextColor(0, 51, 102) // Azul oscuro
    pdf.SetY(40)
    pdf.Cell(0, 20, "CERTIFICADO", 0, 1, "C")

    // Subtítulo
    pdf.SetFont("Arial", "I", 14)
    pdf.SetTextColor(100, 100, 100)
    pdf.SetY(65)
    pdf.Cell(0, 10, "DE COMPLETACION DE CURSO", 0, 1, "C")

    // Línea decorativa
    pdf.SetDrawColor(0, 51, 102)
    pdf.SetLineWidth(1)
    pdf.Line(30, 82, 180, 82)

    // Cuerpo del certificado
    pdf.SetFont("Arial", "", 11)
    pdf.SetTextColor(0, 0, 0)
    pdf.SetY(100)

    pdf.MultiCell(0, 8, fmt.Sprintf(
        "Se certifica que\n\n%s\n\nha completado satisfactoriamente el curso:\n\n%s\n\nco
        nombreAlumno, tituloCurso, calificacion,
    ), 0, "C")

    // Fecha y código
    pdf.SetFont("Arial", "", 10)
    pdf.SetY(200)
    pdf.Cell(0, 8, fmt.Sprintf("Fecha: %s", fechaFinalizacion.Format("02 de January de 20
    pdf.Cell(0, 8, fmt.Sprintf("Código de verificación: %s", codigoUnico), 0, 1, "C")

    // Líneas para firmas
    pdf.SetY(240)
    pdf.Cell(95, 0.1, "", "T", 0, "C")
    pdf.Cell(0, 0.1, "", "T", 1, "C")
    pdf.SetY(250)
    pdf.SetFont("Arial", "", 9)
    pdf.Cell(95, 8, "Firma Instructor", 0, 0, "C")
    pdf.Cell(0, 8, "Sello Institución", 0, 1, "C")

    // Guardar PDF
    filename := fmt.Sprintf("cert_%s_%d.pdf", codigoUnico[:8], time.Now().Unix())
    filepath := filepath.Join(s.CertDir, filename)
```

```
    if err := pdf.OutputFileAndClose(filepath); err != nil {
        return "", err
    }

    return filepath, nil
}

// GuardarCertificadoEnBD registra el certificado en la base de datos
func (s *CertificadoService) GuardarCertificadoEnBD(
    alumnoID, cursoID, inscripcionID int,
    codigoUnico, rutaPDF string,
    calificacionFinal float64,
) (*models.Certificado, error) {
    query := `
        INSERT INTO certificado
        (alumno_id, curso_id, inscripcion_curso_id, codigo_unico, ruta_pdf, calificacion_
        VALUES (?, ?, ?, ?, ?, ?)
    `

    result, err := s.DB.Exec(query, alumnoID, cursoID, inscripcionID, codigoUnico, rutaPD
    if err != nil {
        return nil, err
    }

    id, _ := result.LastInsertId()

    return &models.Certificado{
        ID:                int(id),
        AlumnoID:          alumnoID,
        CursoID:           cursoID,
        InscripcionCursoID: inscripcionID,
        FechaEmision:      time.Now(),
        CodigoUnico:       codigoUnico,
        RutaPDF:           rutaPDF,
        CalificacionFinal:  calificacionFinal,
    }, nil
}
```

services/email_service.go:

```
package services

import (
    "bytes"
    "fmt"
    "html/template"
    "net/smtp"
    "time"

    "course-platform/db"
)

type EmailService struct {
    SMTPHost     string
```

```
    SMTPPort     string
    SMTPUser     string
    SMTPPassword string
    EmailFrom    string
}

func NewEmailService(host, port, user, password, from string) *EmailService {
    return &EmailService{
        SMTPHost:     host,
        SMTPPort:     port,
        SMTPUser:     user,
        SMTPPassword: password,
        EmailFrom:    from,
    }
}

// EnviarEmailAprobacion envía notificación de aprobación con certificado
func (s *EmailService) EnviarEmailAprobacion(
    destinatario, nombreAlumno, tituloCurso string,
    rutaPDF string,
) error {
    asunto := fmt.Sprintf("¡Felicidades! Has aprobado el curso: %s", tituloCurso)

    htmlBody := fmt.Sprintf(`
        <!DOCTYPE html>
        <html>
        <head><meta charset="UTF-8"></head>
        <body style="font-family: Arial, sans-serif; background-color: #f5f5f5; padding:
            <div style="max-width: 600px; margin: 0 auto; background-color: white; paddir
                <h1 style="color: #003366;">¡Felicidades, %s!</h1>
                <p>Has completado y aprobado el curso <strong>%s</strong>.</p>
                <p>Tu certificado de completación está adjunto a este correo.</p>
                <p style="margin-top: 30px; color: #666;">Atentamente,<br>Equipo de Plata
            </div>
        </body>
        </html>
    `, nombreAlumno, tituloCurso)

    return s.enviarEmailConAdjunto(destinatario, asunto, htmlBody, rutaPDF, "aprobacion")
}

// EnviarEmailPlazVencido notifica que el plazo vencio
func (s *EmailService) EnviarEmailPlazoVencido(
    destinatario, nombreAlumno, tituloCurso string,
) error {
    asunto := fmt.Sprintf("Plazo vencido: %s", tituloCurso)

    htmlBody := fmt.Sprintf(`
        <!DOCTYPE html>
        <html>
        <head><meta charset="UTF-8"></head>
        <body style="font-family: Arial, sans-serif; background-color: #fff3cd; padding:
            <div style="max-width: 600px; margin: 0 auto; background-color: white; paddir
                <h2 style="color: #ff6600;">Plazo Vencido</h2>
                <p>Estimado %s,</p>
                <p>El plazo para completar el curso <strong>%s</strong> ha vencido.</p>
```

```
                <p>Si deseas intentarlo nuevamente, ponte en contacto con un instructor.<
                <p style="margin-top: 30px; color: #666;">Atentamente,<br>Equipo de Plata
            </div>
        </body>
        </html>
    `, nombreAlumno, tituloCurso)

    return s.enviarEmail(destinatario, asunto, htmlBody, "plazo_vencido")
}

func (s *EmailService) enviarEmail(destinatario, asunto, htmlBody, tipoEvento string) err
    auth := smtp.PlainAuth("", s.SMTPUser, s.SMTPPassword, s.SMTPHost)

    message := fmt.Sprintf(
        "From: %s\r\nTo: %s\r\nSubject: %s\r\nMIME-Version: 1.0\r\nContent-Type: text/htm
        s.EmailFrom, destinatario, asunto, htmlBody,
    )

    err := smtp.SendMail(
        fmt.Sprintf("%s:%s", s.SMTPHost, s.SMTPPort),
        auth,
        s.EmailFrom,
        []string{destinatario},
        []byte(message),
    )

    // Log en BD
    if err == nil {
        s.registrarLogEmail(destinatario, asunto, tipoEvento, "enviado", "")
    } else {
        s.registrarLogEmail(destinatario, asunto, tipoEvento, "fallido", err.Error())
    }

    return err
}

func (s *EmailService) enviarEmailConAdjunto(destinatario, asunto, htmlBody, rutaPDF, tip
    // Nota: Para adjuntos MIME multipart, se recomienda usar una librería como "github.c
    // Por simplicidad aquí se muestra el flujo, pero en producción usar:
    // e := email.NewEmail()
    // e.From = s.EmailFrom
    // e.To = []string{destinatario}
    // e.Subject = asunto
    // e.HTML = []byte(htmlBody)
    // e.AttachFile(rutaPDF)
    // return e.Send(...)

    err := s.enviarEmail(destinatario, asunto, htmlBody, tipoEvento)
    return err
}

func (s *EmailService) registrarLogEmail(destinatario, asunto, tipoEvento, estado, mensaj
    query := `
        INSERT INTO log_envio_email
        (destinatario, asunto, tipo_evento, fecha_envio, estado, mensaje_error)
        VALUES (?, ?, ?, ?, ?, ?)
```

```
    `

    db.DB.Exec(query, destinatario, asunto, tipoEvento, time.Now(), estado, mensajeError)
}
```

3.7. Middleware de autenticación

middleware/auth.go:

```
package middleware

import (
    "fmt"
    "net/http"
    "strings"

    "github.com/gin-gonic/gin"
    "course-platform/utils"
)

func AuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        if authHeader == "" {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Token requerido"})
            c.Abort()
            return
        }

        parts := strings.Split(authHeader, " ")
        if len(parts) != 2 || parts[0] != "Bearer" {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Formato de token inválido"})
            c.Abort()
            return
        }

        claims, err := utils.VerifyJWT(parts[1])
        if err != nil {
            c.JSON(http.StatusUnauthorized, gin.H{"error": "Token inválido o expirado"})
            c.Abort()
            return
        }

        c.Set("usuarioID", claims.UsuarioID)
        c.Set("rol", claims.Rol)
        c.Next()
    }
}

// RoleMiddleware verifica que el usuario tenga el rol requerido
func RoleMiddleware(rolesPermitidos ...string) gin.HandlerFunc {
    return func(c *gin.Context) {
        rol, exists := c.Get("rol")
        if !exists {
            c.JSON(http.StatusForbidden, gin.H{"error": "Rol no encontrado"})
```

```
            c.Abort()
            return
        }

        rolStr := rol.(string)
        permitido := false
        for _, r := range rolesPermitidos {
            if rolStr == r {
                permitido = true
                break
            }
        }

        if !permitido {
            c.JSON(http.StatusForbidden, gin.H{"error": fmt.Sprintf("Se requiere uno de e
            c.Abort()
            return
        }

        c.Next()
    }
}
```

middleware/role.go:

```
package middleware

// Definir constantes de roles
const (
    RoleAdmin      = "administrador"
    RoleInstructor = "instructor"
    RoleAlumno     = "alumno"
)
```

3.8. Utilidades JWT

utils/jwt.go:

```
package utils

import (
    "fmt"
    "time"

    "github.com/golang-jwt/jwt/v5"
)

type CustomClaims struct {
    UsuarioID int
    Email     string
    Rol       string
    jwt.RegisteredClaims
}
```

```
var jwtSecret = []byte("your-secret-key") // Cargar desde config en producción

func GenerateJWT(usuarioID int, email, rol string) (string, error) {
    claims := CustomClaims{
        UsuarioID: usuarioID,
        Email:     email,
        Rol:       rol,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
        },
    }

    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(jwtSecret)
}

func VerifyJWT(tokenString string) (*CustomClaims, error) {
    token, err := jwt.ParseWithClaims(tokenString, &CustomClaims{}, func(token *jwt.Toker
        if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
            return nil, fmt.Errorf("método de firma inesperado: %v", token.Header["alg"])
        }
        return jwtSecret, nil
    })

    if err != nil {
        return nil, err
    }

    claims, ok := token.Claims.(*CustomClaims)
    if !ok || !token.Valid {
        return nil, fmt.Errorf("claims inválidos")
    }

    return claims, nil
}
```

utils/password.go:

```
package utils

import "golang.org/x/crypto/bcrypt"

func HashPassword(password string) (string, error) {
    hash, err := bcrypt.GenerateFromPassword([]byte(password), bcrypt.DefaultCost)
    return string(hash), err
}

func VerifyPassword(hash, password string) error {
    return bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
}
```

3.9. Handlers principales

handlers/auth.go:

```
package handlers

import (
    "database/sql"
    "net/http"

    "github.com/gin-gonic/gin"
    "course-platform/db"
    "course-platform/models"
    "course-platform/utils"
)

func Register(c *gin.Context) {
    var req models.RegistroRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    // Validar que email sea único
    var existe int
    err := db.DB.QueryRow("SELECT COUNT(*) FROM usuario WHERE email = ?", req.Email).Scar
    if err != nil || existe > 0 {
        c.JSON(http.StatusConflict, gin.H{"error": "Email ya registrado"})
        return
    }

    // Hash de contraseña
    hashPassword, err := utils.HashPassword(req.Password)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error al procesar contrase
        return
    }

    // Insertar usuario
    query := "INSERT INTO usuario (nombre, email, hash_password, rol) VALUES (?, ?, ?, ?)
    result, err := db.DB.Exec(query, req.Nombre, req.Email, hashPassword, req.Rol)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error al crear usuario"})
        return
    }

    id, _ := result.LastInsertId()

    usuario := models.Usuario{
        ID:    int(id),
        Nombre: req.Nombre,
        Email: req.Email,
        Rol:   req.Rol,
        Activo: true,
    }
```

```
    c.JSON(http.StatusCreated, gin.H{"usuario": usuario, "mensaje": "Registro exitoso"})
}

func Login(c *gin.Context) {
    var req models.LoginRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    // Buscar usuario
    var usuario models.Usuario
    var hashPassword string
    err := db.DB.QueryRow(
        "SELECT id, nombre, email, rol, hash_password FROM usuario WHERE email = ?",
        req.Email,
    ).Scan(&usuario.ID, &usuario.Nombre, &usuario.Email, &usuario.Rol, &hashPassword)

    if err == sql.ErrNoRows {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Credenciales inválidas"})
        return
    } else if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error en servidor"})
        return
    }

    // Verificar contraseña
    if err := utils.VerifyPassword(hashPassword, req.Password); err != nil {
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Credenciales inválidas"})
        return
    }

    // Generar JWT
    token, err := utils.GenerateJWT(usuario.ID, usuario.Email, usuario.Rol)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error al generar token"})
        return
    }

    c.JSON(http.StatusOK, models.LoginResponse{
        Token:   token,
        Usuario: usuario,
    })
}
```

handlers/progreso.go:

```
package handlers

import (
    "net/http"
    "strconv"

    "github.com/gin-gonic/gin"
    "course-platform/services"
```

```
)

func ObtenerProgreso(c *gin.Context) {
    inscripcionID, _ := strconv.Atoi(c.Param("inscripcionID"))

    svc := services.NewProgresoService()

    porcentaje, err := svc.CalcularPorcentajeAvance(inscripcionID)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error calculando progreso'
        return
    }

    aprobado, calificacion, err := svc.EvaluarAprobacion(inscripcionID)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error evaluando aprobación
        return
    }

    dentroDelPlazo, err := svc.VerificarPlazo(inscripcionID)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error verificando plazo"})
        return
    }

    c.JSON(http.StatusOK, gin.H{
        "porcentaje_avance":   porcentaje,
        "aprobado":            aprobado,
        "calificacion_final":  calificacion,
        "dentro_del_plazo":    dentroDelPlazo,
    })
}
```

handlers/certificado.go:

```
package handlers

import (
    "net/http"
    "strconv"

    "github.com/gin-gonic/gin"
    "course-platform/db"
    "course-platform/services"
)

func GenerarCertificado(c *gin.Context) {
    inscripcionID, _ := strconv.Atoi(c.Param("inscripcionID"))

    // Obtener datos del alumno y curso
    query := `
        SELECT u.nombre, c.titulo, ic.id, u.id, c.id
        FROM inscripcion_curso ic
        JOIN usuario u ON ic.alumno_id = u.id
        JOIN curso c ON ic.curso_id = c.id
```

```
        WHERE ic.id = ?
    `

    var nombreAlumno, tituloCurso string
    var alumnoID, cursoID int

    err := db.DB.QueryRow(query, inscripcionID).Scan(&nombreAlumno, &tituloCurso, &inscri
    if err != nil {
        c.JSON(http.StatusNotFound, gin.H{"error": "Inscripción no encontrada"})
        return
    }

    // Calcular calificación final
    progresoSvc := services.NewProgresoService()
    calificacion, _ := progresoSvc.CalcularCalificacionFinal(inscripcionID)

    // Generar certificado
    certSvc := services.NewCertificadoService("./certs")
    codigoUnico, _ := certSvc.GenerarCodigoUnico()

    // Crear PDF
    rutaPDF, err := certSvc.GenerarPDFCertificado(nombreAlumno, tituloCurso, time.Now(),
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error generando PDF"})
        return
    }

    // Guardar en BD
    cert, err := certSvc.GuardarCertificadoEnBD(alumnoID, cursoID, inscripcionID, codigoU
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error guardando certificac
        return
    }

    c.JSON(http.StatusCreated, cert)
}

func DescargarCertificado(c *gin.Context) {
    codigoUnico := c.Param("codigo")

    // Validar que el usuario pueda descargar
    usuarioID, _ := c.Get("usuarioID")
    rol, _ := c.Get("rol")

    query := `
        SELECT ruta_pdf, alumno_id FROM certificado WHERE codigo_unico = ?
    `

    var rutaPDF string
    var alumnoID int

    err := db.DB.QueryRow(query, codigoUnico).Scan(&rutaPDF, &alumnoID)
    if err != nil {
        c.JSON(http.StatusNotFound, gin.H{"error": "Certificado no encontrado"})
        return
    }
```

```
    // Validar permisos
    if rol != "administrador" && rol != "instructor" && usuarioID != alumnoID {
        c.JSON(http.StatusForbidden, gin.H{"error": "No tienes permiso para descargar est
        return
    }

    c.File(rutaPDF)
}
```

3.10. Scheduler/Cron para procesamiento automático

cron/scheduler.go:

```
package cron

import (
    "fmt"
    "log"
    "time"

    "github.com/robfig/cron/v3"
    "course-platform/db"
    "course-platform/services"
)

type Scheduler struct {
    cron *cron.Cron
}

func NewScheduler() *Scheduler {
    return &Scheduler{cron: cron.New()}
}

func (s *Scheduler) Start() {
    // Ejecutar diariamente a las 2 AM
    s.cron.AddFunc("0 2 * * *", s.procesarAprobacionesYCertificados)
    s.cron.AddFunc("0 2 * * *", s.procesarPlazoVencido)

    s.cron.Start()
    log.Println("✓ Scheduler iniciado")
}

func (s *Scheduler) Stop() {
    s.cron.Stop()
}

// procesarAprobacionesYCertificados: busca alumnos que aprobaron y genera certificados
func (s *Scheduler) procesarAprobacionesYCertificados() {
    query := `
        SELECT ic.id, u.nombre, u.email, c.titulo, u.id, c.id
        FROM inscripcion_curso ic
        JOIN usuario u ON ic.alumno_id = u.id
        JOIN curso c ON ic.curso_id = c.id
        WHERE ic.estado = 'aprobado'
```

```
            AND NOT EXISTS (
              SELECT 1 FROM certificado WHERE inscripcion_curso_id = ic.id
            )
    `

    rows, err := db.DB.Query(query)
    if err != nil {
        log.Printf("Error en query de aprobaciones: %v", err)
        return
    }
    defer rows.Close()

    progresoSvc := services.NewProgresoService()
    certSvc := services.NewCertificadoService("./certs")
    emailSvc := services.NewEmailService("smtp.gmail.com", "587", "user@gmail.com", "pass

    for rows.Next() {
        var inscripcionID, alumnoID, cursoID int
        var nombreAlumno, email, tituloCurso string

        if err := rows.Scan(&inscripcionID, &nombreAlumno, &email, &tituloCurso, &alumnoI
            log.Printf("Error escaneando fila: %v", err)
            continue
        }

        // Generar certificado
        calificacion, _ := progresoSvc.CalcularCalificacionFinal(inscripcionID)
        codigoUnico, _ := certSvc.GenerarCodigoUnico()
        rutaPDF, _ := certSvc.GenerarPDFCertificado(nombreAlumno, tituloCurso, time.Now()

        _, err := certSvc.GuardarCertificadoEnBD(alumnoID, cursoID, inscripcionID, codigo
        if err != nil {
            log.Printf("Error guardando certificado: %v", err)
            continue
        }

        // Enviar email
        err = emailSvc.EnviarEmailAprobacion(email, nombreAlumno, tituloCurso, rutaPDF)
        if err != nil {
            log.Printf("Error enviando email a %s: %v", email, err)
        }

        log.Printf("✓ Certificado generado y email enviado a %s", email)
    }
}

// procesarPlazoVencido: notifica a alumnos cuyo plazo vencio
func (s *Scheduler) procesarPlazoVencido() {
    query := `
        SELECT ic.id, u.nombre, u.email, c.titulo
        FROM inscripcion_curso ic
        JOIN usuario u ON ic.alumno_id = u.id
        JOIN curso c ON ic.curso_id = c.id
        WHERE ic.estado != 'plazo_vencido'
          AND (
            (ic.fecha_limite IS NOT NULL AND ic.fecha_limite < NOW())
```

```
            OR
            (ic.fecha_limite IS NULL AND c.dias_para_completar IS NOT NULL
             AND DATE_ADD(ic.fecha_inscripcion, INTERVAL c.dias_para_completar DAY) < NOW
          )
    `

    rows, err := db.DB.Query(query)
    if err != nil {
        log.Printf("Error en query de plazo vencido: %v", err)
        return
    }
    defer rows.Close()

    progresoSvc := services.NewProgresoService()
    emailSvc := services.NewEmailService("smtp.gmail.com", "587", "user@gmail.com", "pass

    for rows.Next() {
        var inscripcionID int
        var nombreAlumno, email, tituloCurso string

        if err := rows.Scan(&inscripcionID, &nombreAlumno, &email, &tituloCurso); err !=
            log.Printf("Error escaneando fila: %v", err)
            continue
        }

        // Actualizar estado
        if err := progresoSvc.ActualizarEstadoInscripcion(inscripcionID); err != nil {
            log.Printf("Error actualizando estado: %v", err)
            continue
        }

        // Enviar email
        if err := emailSvc.EnviarEmailPlazoVencido(email, nombreAlumno, tituloCurso); err
            log.Printf("Error enviando email a %s: %v", email, err)
        }

        log.Printf("✓ Notificación de plazo vencido enviada a %s", email)
    }
}
```

3.11. Punto de entrada (`main.go`)

```
package main

import (
    "log"

    "github.com/gin-gonic/gin"
    "course-platform/config"
    "course-platform/cron"
    "course-platform/db"
    "course-platform/handlers"
    "course-platform/middleware"
)
```

```
func main() {
    // Cargar configuración
    cfg := config.LoadConfig()

    // Conectar a BD
    if err := db.InitDB(cfg); err != nil {
        log.Fatalf("Error conectando a BD: %v", err)
    }
    defer db.CloseDB()

    // Iniciar scheduler
    scheduler := cron.NewScheduler()
    scheduler.Start()
    defer scheduler.Stop()

    // Crear router Gin
    router := gin.Default()

    // Rutas públicas
    router.POST("/api/auth/registro", handlers.Register)
    router.POST("/api/auth/login", handlers.Login)

    // Rutas protegidas
    authorized := router.Group("/api")
    authorized.Use(middleware.AuthMiddleware())
    {
        // Cursos
        authorized.GET("/cursos", handlers.ListarCursos)
        authorized.GET("/cursos/:id", handlers.ObtenerCurso)
        authorized.POST("/cursos", middleware.RoleMiddleware(middleware.RoleInstructor, m
        authorized.PUT("/cursos/:id", middleware.RoleMiddleware(middleware.RoleInstructor
        authorized.DELETE("/cursos/:id", middleware.RoleMiddleware(middleware.RoleAdmin),

        // Módulos
        authorized.GET("/cursos/:cursoID/modulos", handlers.ListarModulos)
        authorized.POST("/cursos/:cursoID/modulos", middleware.RoleMiddleware(middleware.

        // Actividades
        authorized.GET("/modulos/:moduloID/actividades", handlers.ListarActividades)
        authorized.POST("/modulos/:moduloID/actividades", middleware.RoleMiddleware(middl

        // Inscripciones
        authorized.POST("/inscripciones", handlers.InscribirAlumno)
        authorized.GET("/mis-inscripciones", handlers.ObtenerMisInscripciones)
        authorized.GET("/inscripciones/:inscripcionID/progreso", handlers.ObtenerProgreso

        // Respuestas
        authorized.POST("/intentos", handlers.CrearIntento)
        authorized.POST("/respuestas", handlers.GuardarRespuesta)

        // Certificados
        authorized.POST("/certificados/:inscripcionID/generar", middleware.RoleMiddleware
        authorized.GET("/certificados/:codigo/descargar", handlers.DescargarCertificado)
    }

    // Iniciar servidor
```

```
    log.Printf("🚀 Servidor iniciando en puerto %s", cfg.ServerPort)
    if err := router.Run(":" + cfg.ServerPort); err != nil {
        log.Fatalf("Error iniciando servidor: %v", err)
    }
}
```

4. FRONTEND CON HTML, CSS, BOOTSTRAP Y JAVASCRIPT

4.1. Estructura de carpetas frontend

```
frontend/
├── index.html
├── login.html
├── registro.html
├── css/
│   ├── style.css
│   └── bootstrap.min.css
├── js/
│   ├── app.js
│   ├── auth.js
│   ├── api.js
│   ├── cursos.js
│   ├── progreso.js
│   └── utils.js
└── pages/
    ├── dashboard-admin.html
    ├── dashboard-instructor.html
    ├── dashboard-alumno.html
    ├── detalle-curso.html
    └── modulos.html
```

4.2. Archivo principal (`index.html`)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Plataforma de Cursos Online</title>
    <link rel="stylesheet" href="css/bootstrap.min.css">
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
    <!-- Navbar -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
        <div class="container-fluid">
            <a class="navbar-brand" href="/">🎓 CourseHub</a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-b
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
```

```
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item" id="nav-usuario" style="display: none;">
                        <span class="nav-link text-light" id="usuario-nombre">Usuario</sp
                    </li>
                    <li class="nav-item">
                        <button class="btn btn-light" id="btn-logout" style="display: nor
                        <a class="nav-link" href="/login.html" id="nav-login">Ingresar</a
                    </li>
                </ul>
            </div>
        </div>
    </nav>

    <!-- Hero Section -->
    <div class="hero-section bg-light py-5" id="hero">
        <div class="container text-center">
            <h1 class="display-4">Bienvenido a CourseHub</h1>
            <p class="lead">Plataforma educativa con certificados y seguimiento personali
            <a href="/login.html" class="btn btn-primary btn-lg">Comenzar ahora</a>
        </div>
    </div>

    <!-- Contenido principal -->
    <div class="container my-5">
        <div id="main-content">
            <!-- Se cargará dinámicamente -->
        </div>
    </div>

    <!-- Scripts -->
    <script src="js/utils.js"></script>
    <script src="js/api.js"></script>
    <script src="js/auth.js"></script>
    <script src="js/app.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.mi
</body>
</html>
```

4.3. Gestión de autenticación (`js/auth.js`)

```
class AuthManager {
    constructor() {
        this.token = localStorage.getItem('token');
        this.usuario = JSON.parse(localStorage.getItem('usuario') || 'null');
    }

    async registrar(nombre, email, password, rol) {
        try {
            const response = await fetch('/api/auth/registro', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ nombre, email, password, rol })
            });

            if (!response.ok) throw new Error('Error en registro');
```

```
        const data = await response.json();
        return data;
    } catch (error) {
        console.error('Error registrando:', error);
        throw error;
    }
}

async login(email, password) {
    try {
        const response = await fetch('/api/auth/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ email, password })
        });

        if (!response.ok) throw new Error('Credenciales inválidas');

        const data = await response.json();

        // Guardar token y usuario
        localStorage.setItem('token', data.token);
        localStorage.setItem('usuario', JSON.stringify(data.usuario));

        this.token = data.token;
        this.usuario = data.usuario;

        return data;
    } catch (error) {
        console.error('Error en login:', error);
        throw error;
    }
}

logout() {
    localStorage.removeItem('token');
    localStorage.removeItem('usuario');
    this.token = null;
    this.usuario = null;
    window.location.href = '/login.html';
}

estaAutenticado() {
    return this.token !== null && this.usuario !== null;
}

getRol() {
    return this.usuario?.rol;
}

getID() {
    return this.usuario?.id;
}

getNombre() {
```

```
        return this.usuario?.nombre;
    }
}

const authManager = new AuthManager();
```

4.4. API Client (`js/api.js`)

```
class APIClient {
    constructor(baseURL = '/api') {
        this.baseURL = baseURL;
    }

    async request(endpoint, options = {}) {
        const url = `${this.baseURL}${endpoint}`;

        const headers = {
            'Content-Type': 'application/json',
            ...options.headers
        };

        if (authManager.token) {
            headers['Authorization'] = `Bearer ${authManager.token}`;
        }

        try {
            const response = await fetch(url, {
                ...options,
                headers
            });

            if (!response.ok) {
                const error = await response.json();
                throw new Error(error.error || `Error HTTP ${response.status}`);
            }

            return await response.json();
        } catch (error) {
            console.error(`Error en ${endpoint}:`, error);
            throw error;
        }
    }

    // Cursos
    async listarCursos() {
        return this.request('/cursos');
    }

    async obtenerCurso(id) {
        return this.request(`/cursos/${id}`);
    }

    async crearCurso(datos) {
        return this.request('/cursos', {
            method: 'POST',
```

```
            body: JSON.stringify(datos)
        });
    }

    // Inscripciones
    async inscribirseEnCurso(cursoID) {
        return this.request('/inscripciones', {
            method: 'POST',
            body: JSON.stringify({ curso_id: cursoID })
        });
    }

    async obtenerMisInscripciones() {
        return this.request('/mis-inscripciones');
    }

    // Progreso
    async obtenerProgreso(inscripcionID) {
        return this.request(`/inscripciones/${inscripcionID}/progreso`);
    }

    // Respuestas
    async crearIntento(inscripcionID, actividadID) {
        return this.request('/intentos', {
            method: 'POST',
            body: JSON.stringify({
                inscripcion_curso_id: inscripcionID,
                actividad_id: actividadID
            })
        });
    }

    async guardarRespuesta(intentoID, preguntaID, opcionID, textoRespuesta) {
        return this.request('/respuestas', {
            method: 'POST',
            body: JSON.stringify({
                intento_actividad_id: intentoID,
                pregunta_id: preguntaID,
                opcion_respuesta_id: opcionID,
                texto_respuesta: textoRespuesta
            })
        });
    }

    // Certificados
    async generarCertificado(inscripcionID) {
        return this.request(`/certificados/${inscripcionID}/generar`, {
            method: 'POST'
        });
    }

    async descargarCertificado(codigo) {
        // Devuelve la URL para descargar
        return `${this.baseURL}/certificados/${codigo}/descargar`;
    }
}
```

```
const apiClient = new APIClient();
```

4.5. Dashboard del alumno (`pages/dashboard-alumno.html`)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi Aprendizaje - CourseHub</title>
    <link rel="stylesheet" href="../css/bootstrap.min.css">
    <link rel="stylesheet" href="../css/style.css">
</head>
<body>
    <!-- Navbar -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
        <div class="container-fluid">
            <a class="navbar-brand" href="/pages/dashboard-alumno.html">🎓 CourseHub</a>
            <div class="navbar-nav ms-auto">
                <span class="nav-link text-light" id="usuario-nombre"></span>
                <button class="btn btn-light" id="btn-logout">Salir</button>
            </div>
        </div>
    </nav>

    <div class="container my-5">
        <!-- Tabs de navegación -->
        <ul class="nav nav-tabs mb-4" role="tablist">
            <li class="nav-item">
                <a class="nav-link active" data-bs-toggle="tab" href="#mis-cursos">Mis Cu
            </li>
            <li class="nav-item">
                <a class="nav-link" data-bs-toggle="tab" href="#catalogo-cursos">Catálogo
            </li>
        </ul>

        <div class="tab-content">
            <!-- Mis Cursos -->
            <div class="tab-pane fade show active" id="mis-cursos">
                <h2>Mis Cursos</h2>
                <div class="row" id="mis-cursos-lista">
                    <!-- Se cargará dinámicamente -->
                </div>
            </div>

            <!-- Catálogo -->
            <div class="tab-pane fade" id="catalogo-cursos">
                <h2>Catálogo de Cursos Disponibles</h2>
                <div class="row" id="catalogo-lista">
                    <!-- Se cargará dinámicamente -->
                </div>
            </div>
        </div>
    </div>
```

```
    <!-- Modal para detalle del curso -->
    <div class="modal fade" id="modalCursoDetalle" tabindex="-1">
        <div class="modal-dialog modal-lg">
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title">Detalle del Curso</h5>
                    <button type="button" class="btn-close" data-bs-dismiss="modal"></but
                </div>
                <div class="modal-body">
                    <h4 id="modal-titulo"></h4>
                    <p id="modal-descripcion"></p>
                    <div id="modal-progreso" style="display: none;">
                        <h6>Progreso</h6>
                        <div class="progress">
                            <div id="barra-progreso" class="progress-bar" style="width: 0
                        </div>
                        <p id="porcentaje-texto"></p>
                        <p id="estado-texto"></p>
                    </div>
                    <div id="modal-modulos">
                        <!-- Se cargan los módulos -->
                    </div>
                </div>
                <div class="modal-footer">
                    <button type="button" id="btn-descargar-cert" class="btn btn-success'
                        □ Descargar Certificado
                    </button>
                    <button type="button" id="btn-inscribirse" class="btn btn-primary" st
                        Inscribirse
                    </button>
                    <button type="button" class="btn btn-secondary" data-bs-dismiss="moda
                </div>
            </div>
        </div>
    </div>

    <script src="../js/utils.js"></script>
    <script src="../js/api.js"></script>
    <script src="../js/auth.js"></script>
    <script src="../js/progreso.js"></script>
    <script>
        // Cargar datos al iniciar
        document.addEventListener('DOMContentLoaded', async () => {
            document.getElementById('usuario-nombre').textContent = authManager.getNombre
            document.getElementById('btn-logout').addEventListener('click', () => authMan

            // Cargar mis cursos
            await cargarMisCursos();
            // Cargar catálogo
            await cargarCatalogo();
        });

        async function cargarMisCursos() {
            try {
                const inscripciones = await apiClient.obtenerMisInscripciones();
```

```
            const container = document.getElementById('mis-cursos-lista');
            container.innerHTML = '';

            for (const ins of inscripciones) {
                const curso = await apiClient.obtenerCurso(ins.curso_id);
                const progreso = await apiClient.obtenerProgreso(ins.id);

                const card = document.createElement('div');
                card.className = 'col-md-4 mb-4';
                card.innerHTML = `
                    <div class="card h-100">
                        <div class="card-body">
                            <h5 class="card-title">${curso.titulo}</h5>
                            <p class="card-text">${curso.descripcion?.substring(0, 10
                            <div class="progress mb-3">
                                <div class="progress-bar" style="width: ${progreso.po
                            </div>
                            <p class="small">Avance: ${progreso.porcentaje_avance.toF
                            <p class="small"><strong>Estado:</strong> ${ins.estado}</
                            <button class="btn btn-sm btn-primary" onclick="verDetall
                                Ver detalles
                            </button>
                        </div>
                    </div>
                `;
                container.appendChild(card);
            }
        } catch (error) {
            console.error('Error cargando mis cursos:', error);
        }
    }

    async function cargarCatalogo() {
        try {
            const cursos = await apiClient.listarCursos();
            const container = document.getElementById('catalogo-lista');
            container.innerHTML = '';

            for (const curso of cursos) {
                const card = document.createElement('div');
                card.className = 'col-md-4 mb-4';
                card.innerHTML = `
                    <div class="card h-100">
                        <div class="card-body">
                            <h5 class="card-title">${curso.titulo}</h5>
                            <p class="card-text text-muted">Tipo: ${curso.tipo}</p>
                            <p class="card-text">${curso.descripcion?.substring(0, 10
                            <p class="small"><strong>Nota mínima:</strong> ${curso.no
                            <button class="btn btn-sm btn-info" onclick="verDetalleCu
                                Ver detalles
                            </button>
                        </div>
                    </div>
                `;
                container.appendChild(card);
            }
```

```
        } catch (error) {
            console.error('Error cargando catálogo:', error);
        }
    }

    async function verDetalleCurso(cursoID, inscripcionID) {
        try {
            const curso = await apiClient.obtenerCurso(cursoID);
            document.getElementById('modal-titulo').textContent = curso.titulo;
            document.getElementById('modal-descripcion').textContent = curso.descripc

            const modalProgreso = document.getElementById('modal-progreso');
            const btnInscribirse = document.getElementById('btn-inscribirse');
            const btnDescargarCert = document.getElementById('btn-descargar-cert');

            btnInscribirse.style.display = 'none';
            btnDescargarCert.style.display = 'none';

            if (inscripcionID) {
                // Usuario ya está inscrito
                const progreso = await apiClient.obtenerProgreso(inscripcionID);
                modalProgreso.style.display = 'block';

                document.getElementById('barra-progreso').style.width = `${progreso.p
                document.getElementById('porcentaje-texto').textContent = `Avance: ${
                document.getElementById('estado-texto').textContent = `Estado: ${prog

                if (progreso.aprobado) {
                    btnDescargarCert.style.display = 'inline-block';
                    btnDescargarCert.onclick = () => descargarCertificado(inscripcion
                }
            } else {
                // Usuario no está inscrito
                modalProgreso.style.display = 'none';
                btnInscribirse.style.display = 'inline-block';
                btnInscribirse.onclick = async () => {
                    try {
                        await apiClient.inscribirseEnCurso(cursoID);
                        alert('¡Inscripción exitosa!');
                        location.reload();
                    } catch (error) {
                        alert('Error en la inscripción: ' + error.message);
                    }
                };
            }

            // Cargar módulos
            const modulosContainer = document.getElementById('modal-modulos');
            // Aquí cargarías los módulos del curso

            const modal = new bootstrap.Modal(document.getElementById('modalCursoDeta
            modal.show();
        } catch (error) {
            console.error('Error:', error);
        }
    }
```

```
        async function descargarCertificado(inscripcionID) {
            try {
                const cert = await apiClient.generarCertificado(inscripcionID);
                const url = apiClient.descargarCertificado(cert.codigo_unico);
                window.open(url, '_blank');
            } catch (error) {
                alert('Error descargando certificado: ' + error.message);
            }
        }
    </script>

    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.mi
</body>
</html>
```

4.6. Página de login (`login.html`)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ingresar - CourseHub</title>
    <link rel="stylesheet" href="css/bootstrap.min.css">
    <style>
        body {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            height: 100vh;
            display: flex;
            align-items: center;
            justify-content: center;
        }
        .login-container {
            background: white;
            border-radius: 10px;
            box-shadow: 0 10px 25px rgba(0,0,0,0.2);
            padding: 40px;
            width: 100%;
            max-width: 400px;
        }
    </style>
</head>
<body>
    <div class="login-container">
        <h1 class="text-center mb-4">□ CourseHub</h1>
        <form id="formLogin">
            <div class="mb-3">
                <label for="email" class="form-label">Email</label>
                <input type="email" class="form-control" id="email" required>
            </div>
            <div class="mb-3">
                <label for="password" class="form-label">Contraseña</label>
                <input type="password" class="form-control" id="password" required>
            </div>
```

```
            <button type="submit" class="btn btn-primary w-100 mb-3">Ingresar</button>
            <p class="text-center">¿No tienes cuenta? <a href="registro.html">Registrarse
        </form>
        <div id="mensaje" style="display: none;" class="alert alert-danger mt-3"></div>
    </div>

    <script src="js/auth.js"></script>
    <script>
        document.getElementById('formLogin').addEventListener('submit', async (e) => {
            e.preventDefault();
            const email = document.getElementById('email').value;
            const password = document.getElementById('password').value;

            try {
                await authManager.login(email, password);
                const rol = authManager.getRol();

                // Redirigir según rol
                if (rol === 'alumno') {
                    window.location.href = '/pages/dashboard-alumno.html';
                } else if (rol === 'instructor') {
                    window.location.href = '/pages/dashboard-instructor.html';
                } else if (rol === 'administrador') {
                    window.location.href = '/pages/dashboard-admin.html';
                }
            } catch (error) {
                document.getElementById('mensaje').textContent = error.message;
                document.getElementById('mensaje').style.display = 'block';
            }
        });
    </script>
</body>
</html>
```

5. SEGURIDAD Y BUENAS PRÁCTICAS

5.1. Sanitización y validación

utils/validator.go:

```
package utils

import (
    "regexp"
    "strings"
)

func SanitizarString(input string) string {
    // Remover caracteres peligrosos
    dangerousChars := regexp.MustCompile(`[<>"'%;()&+]`)
    return dangerousChars.ReplaceAllString(input, "")
}
```

```
func ValidarEmail(email string) bool {
    pattern := `^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`
    matched, _ := regexp.MatchString(pattern, email)
    return matched
}

func ValidarURL(url string) bool {
    pattern := `^https?://[^\s]+$`
    matched, _ := regexp.MatchString(pattern, url)
    return matched
}
```

5.2. Logging centralizado

utils/logger.go:

```
package utils

import (
    "fmt"
    "log"
    "os"
    "time"
)

type Logger struct {
    file *os.File
}

func NewLogger(filename string) (*Logger, error) {
    file, err := os.OpenFile(filename, os.O_CREATE|os.O_APPEND|os.O_WRONLY, 0644)
    if err != nil {
        return nil, err
    }
    return &Logger{file: file}, nil
}

func (l *Logger) Info(message string) {
    l.log("INFO", message)
}

func (l *Logger) Error(message string, err error) {
    l.log("ERROR", fmt.Sprintf("%s - %v", message, err))
}

func (l *Logger) Warn(message string) {
    l.log("WARN", message)
}

func (l *Logger) log(level, message string) {
    logMessage := fmt.Sprintf("[%s] %s - %s\n", time.Now().Format("2006-01-02 15:04:05"),
    l.file.WriteString(logMessage)
    log.Print(logMessage)
}
```

```
func (l *Logger) Close() error {
    return l.file.Close()
}
```

5.3. Validación de entrada en handlers

```
// En handlers/auth.go, validar antes de procesar
func Register(c *gin.Context) {
    var req models.RegistroRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Datos inválidos"})
        return
    }

    // Validar
    if !utils.ValidarEmail(req.Email) {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Email inválido"})
        return
    }

    if len(req.Password) < 8 {
        c.JSON(http.StatusBadRequest, gin.H{"error": "Contraseña muy corta"})
        return
    }

    req.Nombre = utils.SanitizarString(req.Nombre)

    // ... resto del handler
}
```

6. IMPLEMENTACIÓN PASO A PASO: MVP

Paso 1: Preparar entorno

```
# Crear carpeta del proyecto
mkdir course-platform && cd course-platform

# Inicializar módulo Go
go mod init course-platform
go get github.com/gin-gonic/gin
go get github.com/go-sql-driver/mysql
go get github.com/golang-jwt/jwt/v5
go get github.com/jung-kurt/gofpdf/v2
go get github.com/robfig/cron/v3
go get golang.org/x/crypto

# Crear estructura de carpetas
mkdir -p config models handlers services middleware db utils templates cron
```

Paso 2: Base de datos

```
# Crear base de datos
mysql -u root -p < db/migrations.sql

# O ejecutar directamente en MariaDB:
# - Copia el SQL del esquema anterior
# - Ejecuta en cliente MySQL
```

Paso 3: Configuración

```
# Crear archivo .env
cat > .env << EOF
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=tupassword
DB_NAME=course_db
JWT_SECRET=tu-super-secreto-aqui
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=tu_email@gmail.com
SMTP_PASSWORD=tu_app_password
EMAIL_FROM=noreply@courseplatform.com
CERT_DIR=./certs
SERVER_PORT=8080
ENVIRONMENT=development
EOF
```

Paso 4: Implementar backend

- Crear `config/config.go` (como se mostró)
- Crear `db/db.go` con pool de conexiones
- Crear modelos en `models/`
- Crear servicios en `services/`
- Crear handlers en `handlers/`
- Crear middleware en `middleware/`
- Crear `main.go`

Paso 5: Implementar frontend

- Crear estructura HTML en carpeta `frontend/`
- Crear `css/style.css` personalizado
- Crear módulos JavaScript en `js/`
- Crear páginas para cada rol (alumno, instructor, admin)

Paso 6: Ejecutar

```
go run main.go
# Servidor disponible en http://localhost:8080
```

Paso 7: Pruebas

- Registrar usuario
- Login
- Crear curso (como instructor)
- Inscribirse (como alumno)
- Completar actividades
- Generar certificado
- Descargar PDF

7. CARACTERÍSTICAS ADICIONALES

Mejoras futuras:

1. **Búsqueda y filtrado** de cursos por tipo, instructor, fecha
2. **Comentarios y foros** entre alumnos e instructores
3. **Gamificación**: badges, puntos, tablas de clasificación
4. **Análisis avanzados**: reportes de desempeño por instructor
5. **Notificaciones en tiempo real** vía WebSocket
6. **Sincronización con LMS** (Moodle, Blackboard)
7. **Exportación de datos** en formato CSV/Excel
8. **API GraphQL** como alternativa a REST
9. **Docker** para despliegue simplificado
10. **CI/CD** con GitHub Actions

Conclusión

Esta arquitectura proporciona:
✓ **Escalabilidad**: Pool de conexiones, prepared statements, índices de BD
✓ **Seguridad**: JWT, hash de contraseñas, sanitización, logs de auditoría
✓ **Flexibilidad**: Cursos con módulos y actividades heterogéneas
✓ **Automatización**: Scheduler para certificados y notificaciones
✓ **UX moderna**: Bootstrap, JavaScript modular, AJAX
✓ **Mantenibilidad**: Código bien estructurado, separación de capas, documentación

¿Necesitas ayuda con algún aspecto específico de la implementación o tienes preguntas sobre el diseño?